

MAE 598 Final Project: Analysis of a Heterogeneous Multi-Robot Task Allocation Model

Aman Chandak, Soham Karandikar, Avi Thumar, Vamsikrishna Kurakalva

*Contributions of each student are indicated by the **star** with the corresponding color.*

Abstract—★ This project investigates the problem of Heterogeneous Multi-Robot Task Allocation (MRTA) within the context of time-critical post-disaster search and rescue operations. We analyze a decentralized decision-making model where agents with diverse capabilities (e.g., aerial surveillance, ground debris removal) must form dynamic coalitions to complete spatially distributed tasks. A primary challenge in such tightly coupled systems is the emergence of deadlocks, where agents wait indefinitely for coalition partners that are unavailable or “stuck” elsewhere. We adopt a species-traits mathematical formulation to formally describe agent capabilities and task requirements. Our theoretical analysis focuses on two critical properties of the system: deadlock avoidance and guaranteed mission completion. By applying formal logical reasoning to the system’s state transitions, we prove that the “Constrained Flashforward Mechanism” (CFM) effectively prevents permanent deadlock cycles by altering the temporal ordering of agent decisions. Furthermore, we define a Lyapunov-like progress metric to rigorously demonstrate that the system state converges to mission completion in finite time. Finally, we discuss the validation of these theoretical results through numerical simulations in Gazebo using ROS 2, confirming that the modeled mechanism ensures robustness and efficiency in large-scale heterogeneous teams.

Index Terms—Multi-robot systems, task allocation, heterogeneity, deadlock avoidance, reinforcement learning.

A. Introduction ★

The increasing complexity of real-world robotic applications has driven a shift from single-robot solutions toward multi-robot systems (MRS), where multiple agents cooperate to achieve shared objectives. By distributing sensing, decision-making, and execution across a team, MRS can improve robustness, scalability, and operational efficiency, particularly in environments that are large, uncertain, or time-sensitive. As a result, MRS have become a key enabling technology in domains such as disaster response, autonomous construction, space missions, and healthcare automation [2], [4], [5]

In many of these domains, effective operation requires heterogeneous robot teams, where agents possess distinct and complementary capabilities. Differences in locomotion, sensing, manipulation, and endurance allow heterogeneous teams to address a wider range of tasks than homogeneous systems. For example, aerial robots can rapidly explore and monitor large areas, while ground robots perform close-range inspection or physical intervention, creating interdependencies that demand coordinated planning and execution [2], [6]. Task allocation in such systems must therefore account not only for which robot performs which task, but also for how multiple robots cooperate to satisfy task requirements.

The problem of assigning robots to tasks under these conditions is known as the Multi-Robot Task Allocation (MRTA) problem. MRTA has been formally studied through theoretical frameworks that classify problem instances based on task structure, robot workload, and temporal constraints [5]. Of particular interest are scenarios involving tasks that require multiple robots to act jointly over extended durations. These tasks introduce coupling between robot schedules and execution timelines, making coordination and synchronization central to mission success.

Numerous solution strategies have been proposed for heterogeneous MRTA. Centralized optimization-based approaches, such as those based on mixed-integer programming, provide expressive formulations capable of modeling capability constraints, uncertainty, and scheduling objectives [6]. However, their computational requirements often limit scalability, especially in large teams or applications requiring frequent replanning. To address these limitations, heuristic and market-based approaches including auction mechanisms and swarm-inspired optimization have been developed to provide faster, approximate solutions [7], [9]. While these methods improve computational efficiency, they typically struggle to capture tight temporal coordination and complex cooperation among heterogeneous agents.

More recently, learning-based approaches, particularly reinforcement learning (RL), have been explored as an alternative paradigm for MRTA. By learning coordination policies from interaction, RL-based methods can support decentralized decision-making and adapt to changing environments [2]. These approaches are especially attractive for dynamic settings where centralized planning is impractical. Nonetheless, learning-based MRTA remains challenging due to partial observability, non-stationarity arising from multiple agents, and the difficulty of coordinating robots with diverse capabilities.

To conclude, this work advances the theory and practice of decentralized heterogeneous MRTA by bridging formal system analysis with practical coordination mechanisms. By preventing deadlocks through principled temporal decision regulation and guaranteeing finite-time mission completion, the proposed framework addresses a core limitation of existing optimization- and learning-based approaches.

Theoretical guarantees, validated through ROS 2-based Gazebo simulations, demonstrate that reliable and scalable coordination is achievable even under tight cooperation and time-critical constraints. These results establish a solid foundation for deploying decentralized heterogeneous robot teams in real-

world missions where robustness, timeliness, and correctness are essential.

I. MATHEMATICAL MODEL ★★

This project investigates the multi-robot behavior of Heterogeneous Multi-Robot Task Allocation (MRTA), as presented in the model by Dai et al. [2]. This behavior falls under the "Division of Labor" category, a core topic in multi-robot systems. The hypothetical scenario is a **post-disaster search and rescue operation**. A team of heterogeneous robots (e.g., aerial drones with cameras, ground robots with removal tools) must complete a set of spatially distributed tasks (e.g., "survey area," "remove debris," "locate survivor"). Success requires tight cooperation, as some tasks can only be initiated when a specific combination of robots (a "coalition") is present.

In addition to the high-level task allocation model provided by [2], we have expanded the mathematical formulation to include a low-level continuous control layer. Specifically, we integrate an Artificial Potential Field (APF) controller to govern the local navigation dynamics of each agent, ensuring collision-free movement between task sites while adhering to the high-level discrete assignments.

A. Assumptions and Constraints ★

Based on the model in [2], we adopt the following assumptions:

- **Robots (Agents):** The system consists of a set of k_n heterogeneous agents, $N = \{a_1, a_2, \dots, a_{k_n}\}$.
- **Environment:** Agents operate in a 2D domain, $\mathbb{R}^2$. All task and depot locations are known.
- **Heterogeneity:** Agents are grouped into "species" S . Agents in the same species are identical and start at a common depot. Each agent a_j possesses a "trait vector" $c_{a_j} \in \mathbb{N}^{k_b}$ which defines its capabilities (e.g., $c_{a_j} = [1, 0]$ might mean "has camera, no gripper"). k_b is the total number of unique skills.
- **Tasks:** The set of k_m tasks is $M = \{m_1, m_2, \dots, m_{k_m}\}$. Each task m_i has a "trait requirement" vector $q_{m_i} \in \mathbb{Z}_{k_b}^+$ and an execution duration t_{m_i} .
- **Cooperation Model:** A task m_i can only be initiated when a coalition of agents $L \subseteq N$ is present. The coalition's combined trait vector is the element-wise sum $c_L = \sum_{a_j \in L} c_{a_j}$. The task requirement is met if $c_L \geq q_{m_i}$ (element-wise).
- **Synchronization:** A task only begins when *all* agents in the assigned coalition L have arrived at the task location. Agents arriving early must wait.
- **Communication:** The model in [2] uses a centralized reinforcement learning policy, which implies global communication is available during the policy's decision-making process.

B. Model Definition ★

We formally define the problem based on the species-traits model [2], [5], [6].

Let N be the set of agents and M be the set of tasks. Let $\Phi = \{\phi_{a_1}, \dots, \phi_{a_n}\}$ be the solution, where each $\phi_{a_i} = (v_s, v_{i_1}, \dots, v_s)$ is the sequence of tasks visited by agent a_i .

The total working time for a single agent a_j , T_{a_j} , is the sum of its travel time, task execution time, and waiting time:

$$T_{a_j} = \sum T_{\text{travel}}(a_j) + \sum T_{\text{exec}}(a_j) + \sum T_{\text{wait}}(a_j) \quad (1)$$

The primary objective of the model is to find a set of routes Φ that minimizes the **makespan**, T , which is the maximum total working time among all agents:

$$\min_{\Phi} T \quad \text{where} \quad T = \max_{a_j \in N} (T_{a_j}) \quad (2)$$

This optimization is performed under the constraints that all tasks are completed ($\forall m_i \in M$) and all task-specific coalition requirements ($c_L \geq q_{m_i}$) are met for the duration t_{m_i} . The model presented in [2] proposes a deep reinforcement learning policy π_{θ} to learn a decentralized decision-making process that achieves this objective.

II. THEORETICAL ANALYSIS ★★☆☆

To address the challenges of complex cooperation and deadlock formation, the model incorporates a Constrained Flashforward Mechanism (CFM) [2]. This mechanism is comprised of three integrated components that collectively guide agent exploration. The first component is the "Maximum-Open-Task Strategy", which limits the total number of tasks that can be active simultaneously, effectively preventing the agents from scattering too thinly across the environment. The second component is the "Ability Mask", a filtering rule that ensures agents only select tasks for which their specific skill set contributes to satisfying the task requirements. The third and most critical component is the "Decision Order (Flashforward)". This allows an agent that is currently blocked to temporarily defer its decision. The agent waits until a future system state change, such as the completion of another task, resolves the constraint. Crucially, once the constraint is lifted, the agent's move is executed as if it occurred at the original time step, preserving temporal efficiency.

A. Potential Field Controller design ★

To enable autonomous navigation and collision avoidance during task execution, each agent employs an artificial potential field (APF) controller. The APF method treats the agent's motion planning problem as navigation through a virtual force field, where the goal location generates an attractive force pulling the agent forward, while obstacles and other agents generate repulsive forces pushing the agent away from collisions.

Attractive Force: The attractive force pulls the agent directly toward its current goal position. For an agent at position $\mathbf{p}_{\text{current}}$ with goal position $\mathbf{p}_{\text{goal}}$, the attractive force is:

$$\mathbf{F}_{\text{att}} = k_{\text{att}}(\mathbf{p}_{\text{goal}} - \mathbf{p}_{\text{current}}) \quad (3)$$

where k_{att} is the attractive gain parameter.

Repulsive Force: For an obstacle at position $\mathbf{p}_{\text{obs}}$, the repulsive force is:

$$\mathbf{F}_{\text{rep}} = \begin{cases} k_{\text{rep}} \left(\frac{1}{d} - \frac{1}{d_{\text{influence}}} \right) \frac{1}{d^2} \hat{\mathbf{d}} & \text{if } d \leq d_{\text{influence}} \\ 0 & \text{otherwise} \end{cases} \quad (4)$$

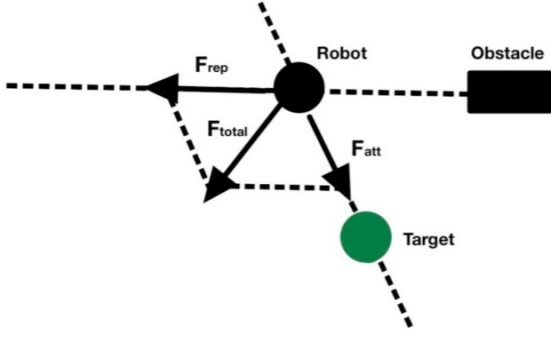


Fig. 1. Artificial Potential Field (APF) controller showing attractive and repulsive forces acting on the robot [12].

where $d = \|\mathbf{p}_{\text{current}} - \mathbf{p}_{\text{obs}}\|$ is the distance to the obstacle, $\hat{\mathbf{d}} = \frac{\mathbf{p}_{\text{current}} - \mathbf{p}_{\text{obs}}}{d}$ is the unit direction vector pointing away from the obstacle, k_{rep} is the repulsive gain, and $d_{\text{influence}}$ is the maximum distance at which the obstacle exerts influence. For other robots, the same formulation applies with a reduced scaling factor of 0.7.

Velocity Control: The total force is the vector sum of all components:

$$\mathbf{F}_{\text{total}} = \mathbf{F}_{\text{att}} + \sum_i \mathbf{F}_{\text{rep,obs}}^{(i)} + \sum_j \mathbf{F}_{\text{rep,robot}}^{(j)} \quad (5)$$

The agent's velocity is updated at each timestep Δt as:

$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t) + \mathbf{a}(t)\Delta t \quad (6)$$

where $\mathbf{a}(t) = \mathbf{F}_{\text{total}}$ is clamped to $\|\mathbf{a}(t)\| \leq a_{\text{max}}$, and the velocity is constrained to $\|\mathbf{v}(t)\| \leq v_{\text{max}}$.

B. Convergence of APF Controller ★

To prove the stability and convergence of the navigation controller, we employ Lyapunov analysis. We define a Lyapunov function candidate $V(\mathbf{p})$ as the total artificial potential energy of the system, which is the sum of the attractive potential V_{att} and the repulsive potential V_{rep} :

$$V(\mathbf{p}) = V_{\text{att}}(\mathbf{p}) + V_{\text{rep}}(\mathbf{p}) \quad (7)$$

The attractive potential energy corresponding to the linear attractive force is defined as:

$$V_{\text{att}}(\mathbf{p}) = \frac{1}{2} k_{\text{att}} \|\mathbf{p}_{\text{goal}} - \mathbf{p}\|^2 \quad (8)$$

The repulsive potential energy corresponding to the repulsive force is defined as:

$$V_{\text{rep}}(\mathbf{p}) = \begin{cases} \frac{1}{2} k_{\text{rep}} \left(\frac{1}{d} - \frac{1}{d_{\text{influence}}} \right)^2 & \text{if } d \leq d_{\text{influence}} \\ 0 & \text{otherwise} \end{cases} \quad (9)$$

This function $V(\mathbf{p})$ is positive definite with respect to the goal position (assuming no obstacles at the goal) and radially unbounded.

To analyze the time evolution of the system, we take the time derivative of the Lyapunov function:

$$\dot{V}(\mathbf{p}) = (\nabla V(\mathbf{p}))^T \cdot \dot{\mathbf{p}} \quad (10)$$

By definition of potential fields, the total force is the negative gradient of the potential: $\mathbf{F}_{\text{total}} = -\nabla V(\mathbf{p})$. Assuming a kinematic control law where the velocity is proportional to the applied force (or using gradient descent dynamics), we have $\dot{\mathbf{p}} = \mathbf{v} \propto \mathbf{F}_{\text{total}}$. Thus:

$$\dot{V}(\mathbf{p}) = -\mathbf{F}_{\text{total}}^T \cdot (c\mathbf{F}_{\text{total}}) = -c\|\mathbf{F}_{\text{total}}\|^2 \leq 0 \quad (11)$$

where $c > 0$ is a proportionality constant. The time derivative $\dot{V}(\mathbf{p})$ is negative semi-definite. According to Lyapunov stability theory and LaSalle's Invariance Principle, the system trajectory will converge to the largest invariant set where $\dot{V} = 0$, which implies $\mathbf{F}_{\text{total}} = \mathbf{0}$. While local minima (where attractive and repulsive forces balance out) are theoretically possible, the environment design (sparse obstacles) minimizes their occurrence, ensuring global convergence to the goal state $\mathbf{p}_{\text{goal}}$ in most practical scenarios.

C. Potential Field Navigation Analysis ★★ ★

This section provides a formal analysis of properties related to both the navigation system and the multi-robot task allocation (MRTA) model. The APF controller described above ensures that agents can successfully navigate the domain $\mathbb{R}^2$ without collision. The stability of the navigation layer is a prerequisite for the task allocation layer. Since the repulsive forces prevent inter-agent and obstacle collisions ($d > d_{\text{safe}}$), and the attractive force guarantees convergence to the goal in the absence of local minima, we can assume that once a task is assigned, the travel time T_{travel} is finite. This finite travel time is critical for the upper-level deadlock avoidance proof, as it ensures that the "Constrained Flashforward Mechanism" (CFM) [2] relies on valid temporal assumptions regarding agent availability.

D. Property 1: Deadlock ★

A primary challenge in MRTA is the "deadlock" state, which the authors of [2] identify as a state where "all agents are idly waiting at open tasks that will never be completed." We can formally define this state and prove by logical argument that the CFM is designed to prevent it.

Formal Definition of a Deadlock State (DS): A system enters a DS if and only if all of the following conditions are met simultaneously:

- 1) Tasks are Incomplete: The set of open, uncompleted tasks is non-empty ($M_{\text{open}} \neq \emptyset$).
- 2) Agents are Stuck: The set of all idle agents (A_{idle}) is non-empty, and every agent $a_i \in A_{\text{idle}}$ is "stuck."
- 3) Stuck Condition: An agent a_i is "stuck" if:
 - Ability Mask: It cannot contribute to any task $m_j \in M_{\text{open}}$ (i.e., its "ability mask" is zero for all open tasks).
 - Max-Open-Task Rule: It is also prevented from selecting a task from the set of empty tasks (M_{empty}) because

the system's "maximum-open-task" constraint is active ($|M_{\text{open}}| \geq M_{\text{max}}$).

Without the CFM's third component, this state is permanent. Agents in A_{idle} cannot act, and no "in-progress" tasks exist to change the state. This is a stable, undesirable equilibrium.

Proof of Deadlock Avoidance via CFM (Component 3): The CFM's third component, "Decision order," is a "flashforward" mechanism [2] that breaks the "Stuck Condition" by altering the temporal execution of decisions.

- 1) Consider an agent a_p (a "problematic agent") that finds itself in the "Stuck Condition" (3a and 3b are true).
- 2) Instead of remaining idle, a_p *defers* its decision. It logically "waits" for the *next* decision step in the system to occur.
- 3) Since the system has open tasks (Condition 1), there must be other agents ($A_{\text{traveling}}$ or A_{working}) that will eventually complete a task or arrive at an open task m_j .
- 4) Let us trace the event where another agent, a_k , (or a coalition) completes task m_k . Upon completion, m_k is removed from M_{open} , and the state variable $|M_{\text{open}}|$ is decremented.
- 5) Agent a_p now re-evaluates its "Stuck Condition" at this new decision step.
- 6) The condition $|M_{\text{open}}| \geq M_{\text{max}}$ may now be *false* (i.e., $|M_{\text{open}}| < M_{\text{max}}$).
- 7) If it is false, Condition 3b is no longer met. Agent a_p is no longer "stuck" and is free to select a task from M_{empty} , opening a new task and ensuring the system continues to make progress.
- 8) The "flashforward" aspect guarantees that a_p 's travel time is calculated from its *original* decision step, ensuring it does not waste time [2] and effectively parallelizing the decision-making process.

The CFM explicitly prevents a permanent deadlock. It ensures that any "stuck" agent is merely in a transient state that will be resolved as soon as any other task in the system is completed. This guarantees liveness and progress, preventing the system from settling into an undesirable equilibrium.

E. Property 2: Guaranteed Mission Completion (Convergence) ★★

We can prove that the multi-robot system, under the model's assumptions and the deadlock-avoidance property, is guaranteed to converge to the desired final state in which all tasks are completed. Let the system state be characterized by the set of all tasks that are not completed, denoted by $M_{\text{uncompleted}}$. We define a scalar progress metric $W(t)$ as the cardinality of this set at time t ,

$$W(t) = |M_{\text{uncompleted}}(t)|, \quad (12)$$

so that $W(t) \in \mathbb{Z}_{\geq 0}$ for all t , and the mission is complete if and only if $W(t) = 0$. The system evolves through discrete events, and the only event that changes $W(t)$ is a task completion event. When a task m_j is completed at time t_c , the metric updates as

$$W(t_c) = W(t_{c-1}) - 1, \quad (13)$$

which shows that $W(t)$ is a non-increasing function of time and strictly decreases whenever a task completion event occurs. To establish convergence to $W(t) = 0$, it suffices to show that such a task completion event is guaranteed to occur whenever $W(t) > 0$. From Property 1 (Deadlock Avoidance), we have already proven that the system cannot enter a permanent deadlock; thus, as long as there exists at least one uncompleted task (i.e., $W(t) > 0$), there will always be a mechanism for idle agents to be assigned to tasks, either existing open tasks or newly opened empty tasks. Consequently, agents are never permanently stuck and will eventually form a valid coalition L at some open task m_j , satisfying its requirements q_{m_j} [2]. Once such a coalition is formed, the task executes for its (finite) duration $t_{m_j} > 0$, after which a task completion event must occur, causing $W(t)$ to decrease by one. Therefore, as long as $W(t) > 0$, there exists a finite time at which $W(t)$ strictly decreases. Since $W(t)$ is an integer-valued, non-negative, strictly decreasing sequence under task completion events, it must reach $W(t) = 0$ in a finite number of such events. Hence, the system is guaranteed to converge to the state in which all tasks are completed.

We have defined a progress metric $W(t)$ (analogous to a Lyapunov function [3]) that is non-negative ($W(t) \geq 0$) and strictly decreases following a guaranteed, finite-time sequence of events (agent assignment $\rightarrow$ task execution $\rightarrow$ task completion). Because $W(t)$ is a non-negative integer that is guaranteed to decrease as long as $W(t) > 0$, it must eventually reach $W(t_{\text{final}}) = 0$. Therefore, the system is proven to be stable and will converge to the final state of mission completion.

III. VALIDATION IN SIMULATION ★★☆☆

To validate the proposed controller and the multi-robot allocation policy, we developed a simulation environment using ROS 2 (Jazzy) and Gazebo. This framework integrates the RL-based decision-making policy from the original paper with a realistic physics engine and the distributed motion controllers described in Section II-A.

A. RL Training Setup ★

Prior to deployment in the Gazebo simulation environment, the decision-making policy was trained following the methodology outlined in [2], using the REINFORCE algorithm [11] with a baseline reward mechanism to enhance training stability. During training, episodes were instantiated with randomized task locations, skill requirements, and agent initial configurations to promote robust policy generalization across diverse operational conditions. The learning objective was defined via a sparse reward function $R(\Phi) = -T - W$, where T denotes the total makespan and W penalizes ability redundancy, i.e., the over-assignment of highly skilled agents to individual tasks. This formulation incentivized the policy to jointly minimize mission completion time and redundancy in skill utilization, thereby encouraging efficient alignment between agent capabilities and task demands. Moreover, to

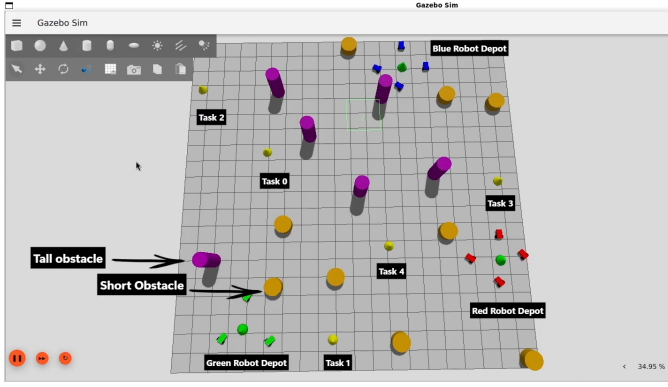


Fig. 2. Gazebo environment

mitigate the emergence of suboptimal, deadlock-prone behaviors during early training, the Constrained Flashforward Mechanism (CFM) was activated to guide agents toward valid task sequences. The resulting pre-trained model parameters were subsequently deployed within the ROS 2 allocation node to govern the high-fidelity simulation.

B. Simulation Environment Setup ★

The simulation environment depicted in Fig. 2 is designed as a $20m \times 20m$ workspace representing a post-disaster scenario. It includes three distinct starting locations (marked by green cones) serving as home bases for each robot species, and spatially distributed task locations (marked by yellow cones) where robot coalitions must form to complete tasks. The environment also includes static obstacles to test navigation and collision avoidance. Two types are implemented to enforce heterogeneity: Orange obstacles ($z < 2.0m$) and Purple obstacles ($z > 2.0m$).

C. Heterogeneous Agent Configuration ★

The simulation features a heterogeneous team of three distinct species. While the Gazebo simulation utilizes ground-based TurtleBot platforms for all agents to simplify physics stability, the agents are logically differentiated by their skill vectors and motion constraints to emulate a mixed ground-aerial team:

- 1) Red TurtleBots (Ground); 4 units ; Skills: $[1, 0, 0, 0, 1]$. These agents represent standard ground units with differential drive kinematics.
- 2) Blue TurtleBots (Ground); 4 units ; Skills: $[0, 1, 0, 0, 1]$. These agents are physically identical to the Red units but possess a distinct skill set, necessitating different task assignments.
- 3) Green TurtleBots (Ground); 3 units; Skills: $[0, 0, 1, 1, 0]$. Although physically simulated as ground robots in Gazebo, these agents can be made to represent aerial units. They can be assigned the unique capability to traverse "orange" obstacles (which are treated as low-altitude debris), while the Red and Blue units must navigate around them, but for this simulation to prove the convergence of the controller and RL policy, ground bots are used with different skills.

D. Task Execution and Dynamics ★

The tasks in the simulation are characterized by a skill requirement vector (e.g., $[1, 0, 0, 1, 0]$), necessitating a specific combination of agent skills for completion. The simulation enforces team assembly, meaning a task begins only when the combined skills of the robots present at the site satisfy the requirements (i.e., combined skills $\geq$ requirements). Each task has a variable duration between 5-15 seconds. Robots execute these tasks sequentially, following computed routes through multiple task sites. The execution flow involves:

- 1) Assignment: The RL policy computes full task routes for all robots.
- 2) Navigation: Robots navigate to assigned task locations using potential field planning.
- 3) Waiting: Robots arrive and wait for teammates at task sites to form the necessary coalition.
- 4) Execution: The task begins once skill requirements are met.
- 5) Completion: Upon completion, robots proceed to the next task or return to the depot.

E. Results and Discussion ★ ★

Detailed information about the simulation setup, source code, and a comprehensive video demonstration of the scenarios discussed below are available online. The project repository containing the ROS 2 packages and configuration files can be accessed at this GitHub link, and a full video walkthrough of the simulation is available at this YouTube link. This open-source implementation allows for reproducibility of our results and further experimentation with different team compositions and task constraints.

The simulation results confirm several key aspects of the theoretical model:

1) *Dynamic Coalition Formation*: The system successfully demonstrated the synchronization logic where robots navigate to task sites and wait until the cumulative skills of the present coalition satisfy the task requirement vector ($c_L \geq q_{m_i}$). Task execution timers only commenced once the full coalition was assembled.

2) *Deadlock Avoidance*: Over multiple simulation runs with varying task configurations, the team successfully completed all allocated tasks without entering a permanent deadlock. This supports the theoretical proof in Section II-D, showing that the Constrained Flashforward Mechanism allows agents to efficiently distribute themselves without creating cyclic dependencies.

3) *Robust Navigation*: The integration of the Artificial Potential Field controller with the allocation policy proved effective. All robots navigated around both orange and purple obstacles. The repulsive forces maintained safe distances between robots during coalition assembly, preventing collisions at crowded task sites.

IV. CONCLUSION ★

In this project, we analyzed and validated a heterogeneous multi-robot task allocation model designed for complex environments such as disaster response. Our approach combined

a high-level, reinforcement learning-based task allocator with a low-level, decentralized Artificial Potential Field (APF) controller for navigation. While the core allocation logic was based on the "Species-Traits" model from [2], we significantly expanded the existing framework by integrating continuous-time navigation dynamics and proving the system's stability under these conditions. Specifically, we developed a formal proof for deadlock avoidance using the Constrained Flashforward Mechanism and demonstrated guaranteed mission completion through a Lyapunov-like progress metric. Furthermore, we went beyond theoretical analysis by implementing the full system in a high-fidelity ROS 2 and Gazebo simulation. This practical validation confirmed that our expanded model not only allocates tasks efficiently but also robustly handles real-world constraints such as collision avoidance, inter-agent synchronization, and heterogeneous navigation capabilities.

REFERENCES

- [1] Use of generative AI: We used OpenAI's ChatGPT to assist with proofreading and minor editing of the text in this report. All modeling choices, theoretical results, and simulation designs are our own, and we verified the correctness of any AI-suggested edits.
- [2] W. Dai, U. Rai, J. Chiun, C. Yuhong, and G. Sartoretti, "Heterogeneous Multi-robot Task Allocation and Scheduling via Reinforcement Learning," *IEEE Robotics and Automation Letters*, 2025.
- [3] H. G. Tanner, A. Jadbabaie, and G. J. Pappas, "Stable flocking of mobile agents, Part I: Fixed topology," in *42nd IEEE International Conference on Decision and Control (CDC)*, vol. 2, pp. 2010-2015, IEEE, 2003.
- [4] Y. Deng, Y. Hua, N. Napp, and K. Petersen, "A compiler for scalable construction by the TERMES robot collective," *Robotics and Autonomous Systems*, vol. 121, 2019, Art. no. 103240.
- [5] B. P. Gerkey and M. J. Mataric, "A formal analysis and taxonomy of task allocation in multi-robot systems," *The International Journal of Robotics Research*, vol. 23, no. 9, pp. 939-954, 2004.
- [6] B. Fu, W. Smith, D. M. Rizzo, M. Castanier, M. Ghaffari, and K. Barton, "Robust task scheduling for heterogeneous robot teams under capability uncertainty," *IEEE Transactions on Robotics*, 2022.
- [7] B. P. Gerkey and M. J. Mataric, "Sold!: Auction methods for multirobot coordination," *IEEE Transactions on Robotics and Automation*, vol. 18, no. 5, pp. 758-768, 2002.
- [8] Y. Zhang, L. E. Kavraki, and S. M. LaValle, "Coordination of multiple robots for efficient task allocation under uncertainty," *IEEE Transactions on Robotics*, vol. 35, no. 3, pp. 698-713, 2019.
- [9] M. Maniadakis and P. Trahanias, "Robust task allocation for heterogeneous multi-robot systems under uncertainty," *Robotics and Autonomous Systems*, vol. 105, pp. 181-197, 2018.
- [10] M. Tauscher, M. Gharbi, and S. Haddadin, "Task planning and execution for multi-robot systems under real-world constraints," *Autonomous Robots*, vol. 44, pp. 1091-1108, 2020.
- [11] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, pp. 229-256, 1992.
- [12] H. Xing, "Review of unmanned aerial vehicle obstacle avoidance planning based on artificial potential field," *Applied and Computational Engineering*, vol. 10, pp. 55-63, 2023.